



MMA093 Rigid Body Dynamics

Assignment 4

Mandatory Part

Ramkumar Huleppa Munavalli

Email : huleppa@student.chalmers.se

Chalmers University of Technology

MMA093 Rigid Body Dynamics

December 10, 2025

Problem 1: Rotation Matrix

```
clear; clc; close all;
```

```
% Problem 1 (Rotation Matrix)
```

```
syms alpha beta gamma real
```

```
Rx = [1, 0, 0; 0, cos(gamma), sin(gamma); 0, -sin(gamma), cos(gamma)];
```

```
Ry = [cos(alpha), 0, -sin(alpha); 0, 1, 0; sin(alpha), 0, cos(alpha)];
```

```
Rz = [cos(beta), sin(beta), 0; -sin(beta), cos(beta), 0; 0, 0, 1];
```

```
Rotation_Matrix = Rx * Rz * Ry;
```

```
disp('The Rotation Matrix:');
```

```
disp(Rotation_Matrix);
```

The Rotation Matrix:

```
[ cos(alpha)*cos(beta), sin(beta), -cos(beta)*sin(alpha)]  
[ sin(alpha)*sin(gamma) - cos(alpha)*cos(gamma)*sin(beta), cos(beta)*cos(gamma), cos(alpha)*sin(gamma) + cos(gamma)*sin(alpha)*sin(beta)]  
[ cos(gamma)*sin(alpha) + cos(alpha)*sin(beta)*sin(gamma), -cos(beta)*sin(gamma), cos(alpha)*cos(gamma) - sin(alpha)*sin(beta)*sin(gamma)]
```

Problem 2: Angular velocity vector

```
syms alpha_dot beta_dot gamma_dot X Y Z x real

% Global_axis = [X; Y; Z];
X = [1 ; 0; 0];
Y = [0 ; 1; 0];
Z = [0 ; 0; 1];

Ry_hat = Ry';

y_hat = Y; % global Y
z_hat = simplify(Ry_hat * Z); % R_y(alpha) * z
x_hat = simplify(Ry_hat * Rz' * X); % R_y(alpha)*R_z(beta)*x

% angular velocity
omega = simplify(alpha_dot*y_hat + beta_dot*z_hat + gamma_dot*x_hat);

% show result
omega = simplify(omega);
disp(omega);
disp('Components (omega_x, omega_y, omega_z):');
omega_x = simplify(omega(1));
omega_y = simplify(omega(2));
omega_z = simplify(omega(3));
omega_x, omega_y, omega_z
```

```
beta_dot*sin(alpha) + gamma_dot*cos(alpha)*cos(beta)
alpha_dot + gamma_dot*sin(beta)
beta_dot*cos(alpha) - gamma_dot*cos(beta)*sin(alpha)
```

Problem 3: Velocity & Acceleration of point P

```
clear; clc; close all;
```

Parameters (numerical) -----

```
alpha_dot_val = pi/20; % rad/s
beta_dot_val  = pi/10;  % rad/s
gamma_dot_val = -pi/4;  % rad/s

% geometry (m)
rA_num = [0; 0.8; 0]; % position of A in global frame
AB_num = [0.7; 0; 0]; % A->B expressed in arm local
rhook_num = [0.1; -0.45; 0]; % B->P in hook local

t_eval = 10; % time to evaluate
```

Symbolic setup

```

syms t real

% Define time-dependent angles (linear in time)
alpha = alpha_dot_val * t;
beta  = beta_dot_val  * t;
gamma = gamma_dot_val * t;

% Elementary rotation matrices (use your exact sign convention)
Rx = [1, 0, 0;
      0, cos(gamma), sin(gamma);
      0, -sin(gamma), cos(gamma)];

Ry = [cos(alpha), 0, -sin(alpha);
      0,          1, 0;
      sin(alpha), 0, cos(alpha)];

Rz = [cos(beta), sin(beta), 0;
      -sin(beta), cos(beta), 0;
      0,          0,          1];

% Compose transforms consistent with your earlier code:
% Q = Ry * Rz (rotation applied to AB)
% Rfull = Q * Rx (full rotation applied to rho_k)
Q = simplify(Ry' * Rz');
Rfull = simplify(Q * Rx');

```

Position of P (symbolic)

$rP(t) = rA + Q*AB + Rfull * rho_k$ Use numeric geometry but symbolic rotations: convert AB and rho_k to symbolic columns

```

AB = sym(AB_num);
rho_k = sym(rhook_num);
rA = sym(rA_num);

rP_sym = simplify(rA + Q*AB + Rfull*rhook);
% extract components
Xcomp = simplify(rP_sym(1));
Ycomp = simplify(rP_sym(2));
Zcomp = simplify(rP_sym(3));

% show symbolic forms (optional)
% disp('rP symbolic components:'); disp(Xcomp); disp(Ycomp); disp(Zcomp);

```

Angular velocity (symbolic) expressed in global frame $\omega = \alpha_{\dot{}} \hat{y} + \beta_{\dot{}} (R_y \hat{z}) + \gamma_{\dot{}} (R_y R_z \hat{x})$

```
Xvec = [1;0;0]; Yvec = [0;1;0]; Zvec = [0;0;1];
```

```
z_hat = simplify(Ry * Zvec);           % axis for beta (in global)
x_hat = simplify(Ry * (Rz * Xvec));     % axis for gamma (in global)

omega_sym = alpha_dot_val * Yvec + beta_dot_val * z_hat + gamma_dot_val * x_hat;
omega_sym = simplify(omega_sym);
```

Velocity (symbolic) = time derivative of position Acceleration (symbolic) = second time derivative

```
vP_sym = simplify( diff(rP_sym, t) );
aP_sym = simplify( diff(vP_sym, t) );
```

Numeric evaluation at $t = t_{\text{eval}}$

evaluate numeric values

```
rP_num = double( subs(rP_sym, t, t_eval) );
vP_num = double( subs(vP_sym, t, t_eval) );
aP_num = double( subs(aP_sym, t, t_eval) );
omega_num = double( subs(omega_sym, t, t_eval) );
```

Display results

Display results

```
fprintf('\nResults evaluated at t = %.3f s\n', t_eval);
fprintf('-----\n');

fprintf('Position r_P (m):\n');
disp(rP_num);

fprintf('Velocity v_P (m/s):\n');
disp(vP_num);

fprintf('Acceleration a_P (m/s^2):\n');
disp(aP_num);
```

Results evaluated at t = 10.000 s

Position r_P (m):

0.4500

0.8000

0.8000

Velocity v_P (m/s):

0.1257

-0.6048

-0.0707

Acceleration a_P (m/s^2):

-0.2887

0

-0.3208

Problem 4: Model of Robot

A robot was modeled in ADAMS following the instructions given. Translational velocity and Acceleration components were measured for P point and the results were compared with the one obtained in problem 3.

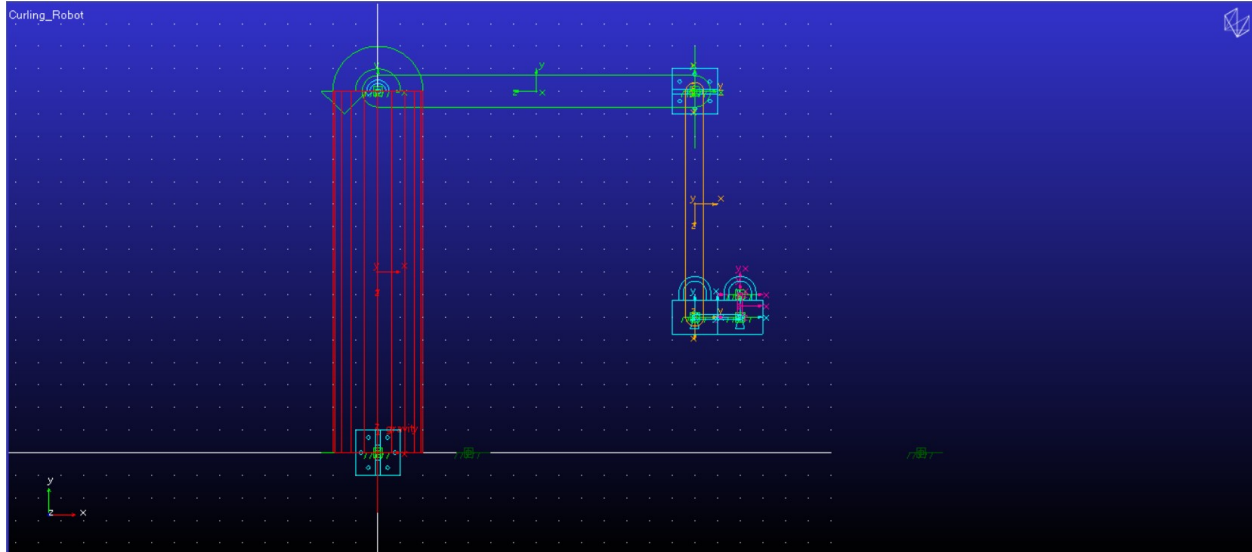


Figure 1: Model of Robot

The angular velocity, Velocity of point P and Acceleration of Point P is determined.

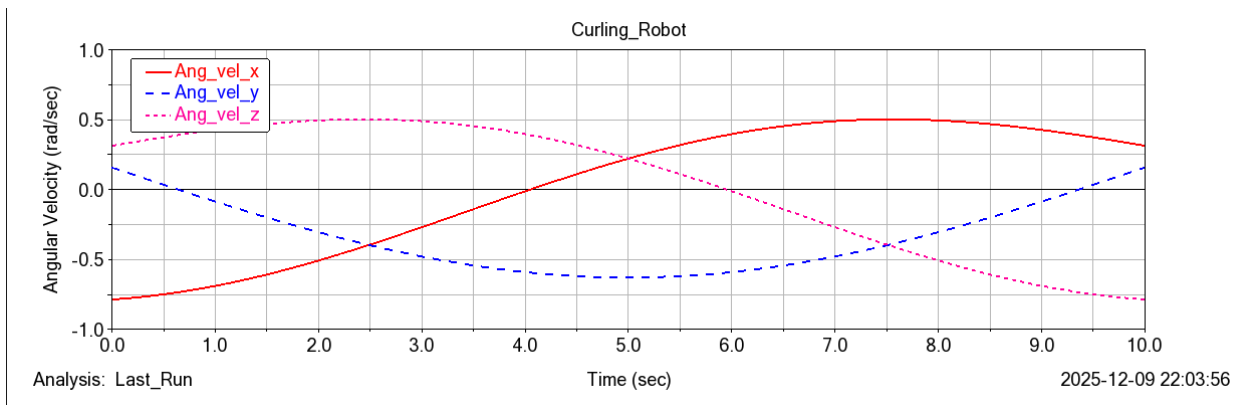


Figure 2: Angular velocity(rad/sec)

	X frame	Y frame	Z frame
ADAMS	0.3142	0.1571	-0.7854
analytical	0.3142	0.1571	-0.7854

Table 1: Angular velocity (rad/sec)

In ADAMS and the analytical solution for angular velocity of point P are similar and analytical solution is verified. Refer table 1.

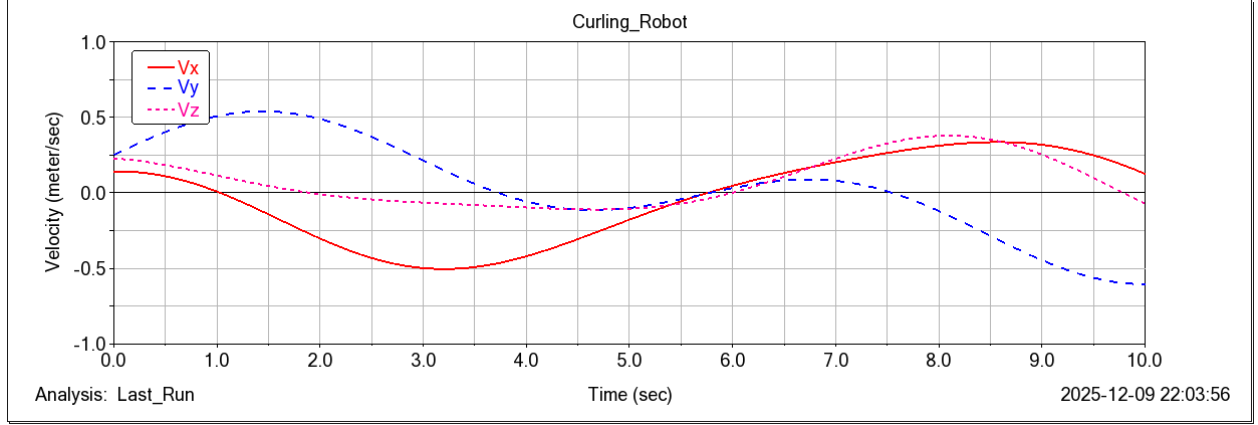


Figure 3: Velocity at Point P(m/s)

	X frame	Y frame	Z frame
ADAMS	0.1257	-0.6041	-0.0707
analytical	0.1257	-0.6048	-0.0707

Table 2: Velocity of P(meter/sec)

In ADAMS and the analytical solution for velocity of point P are very close and analytical solution is verified. Refer table 2

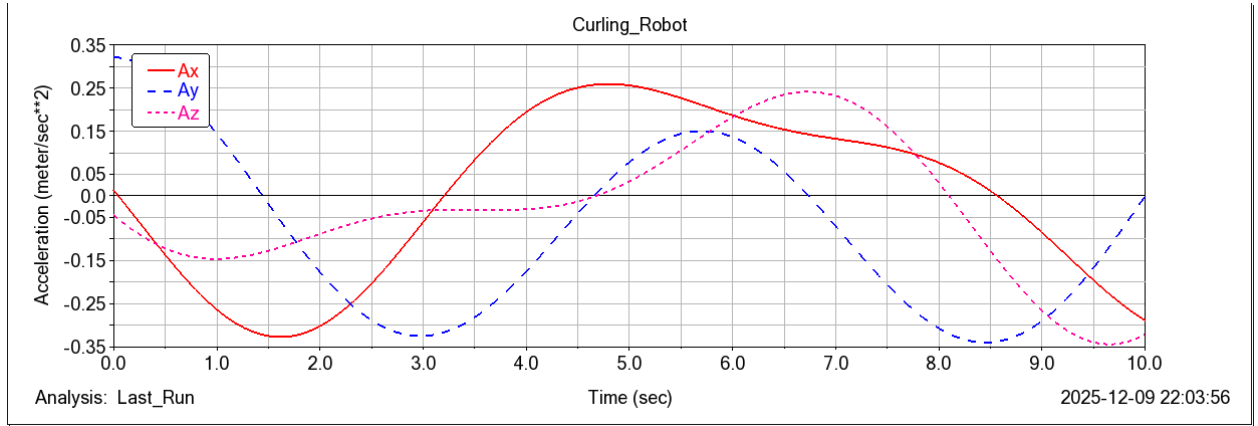


Figure 4: Acceleration at point P (m/s^2)

	X frame	Y frame	Z frame
ADAMS	-0.2857	0.0069	-0.3258
analytical	-0.2887	0	-0.3208

Table 3: Acceleration of P (meter/sec²)

In ADAMS and the analytical solution for acceleration of point P are very close and analytical solution is verified. Refer table 3.

Problem 5: Curling stone

The modeled robot is made to lift and throw the curling stone along the ice strip.

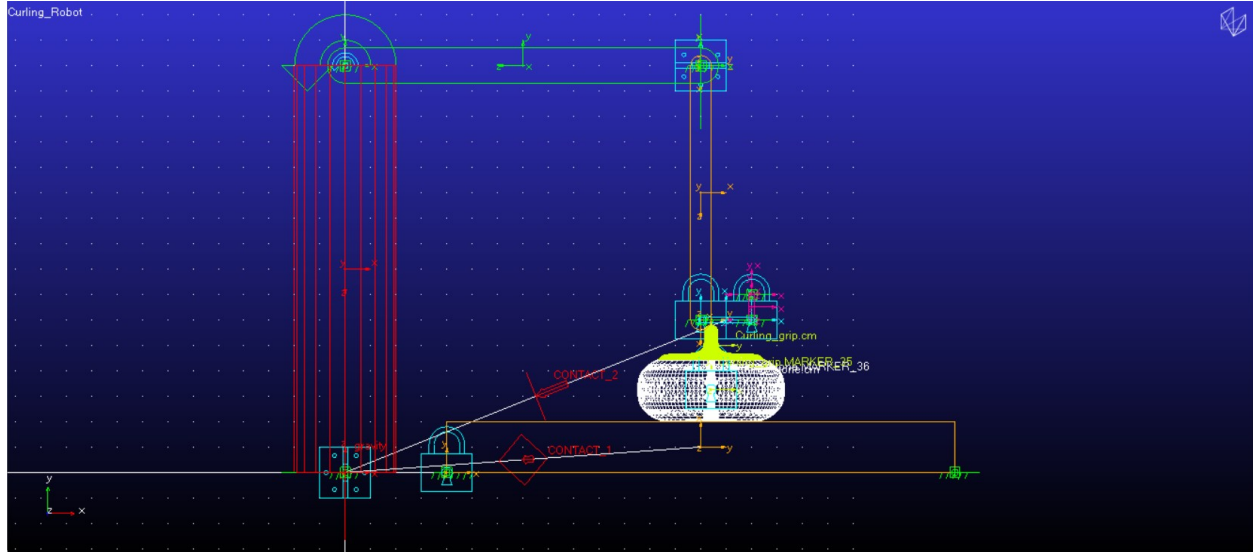


Figure 5: Model of the curling stone

	Swing up the hook		Low down the arm		Swing back the hook		Lift the curling stone		Throw the curling stone	
Time	0 s	0.5 s	1 s	1.5 s	2 s	2.5 s	3 s	3.5 s	3.8 s	4 s
α (deg)	0	0	0	0	0	-10	-10	-10	-10	-10
β (deg)	0	0	-7	-7	-7	-7	3.5	3.5	0	0
γ (deg)	30	30	30	30	30.5	30.5	0	0	0	0

Figure 6: Control Sequence

Tabulated control sequence is shown above Fig 6.

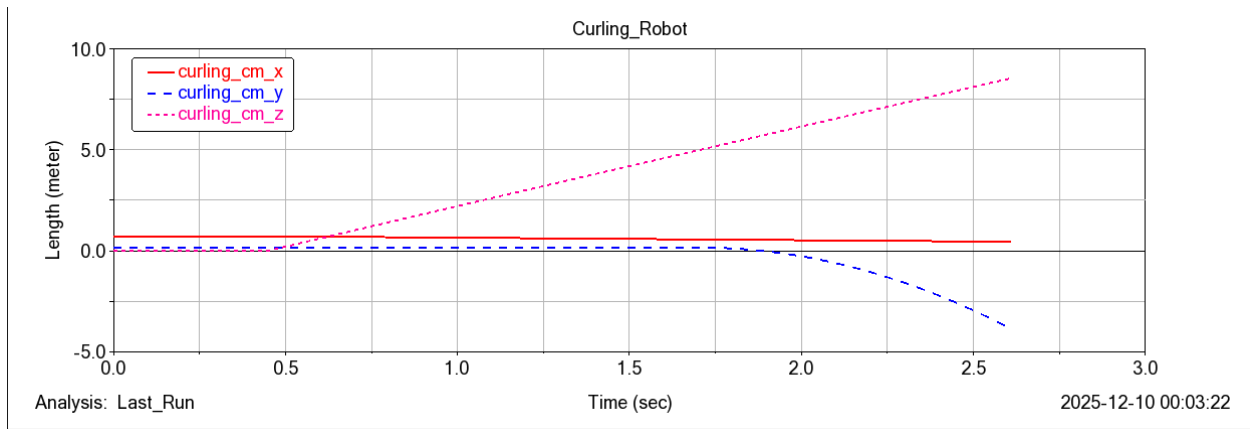


Figure 7: Curling stone position in XYZ

The curling stone position in X is 0.451m, in Y is -3.789m and in Z is 8.461m is shown above Fig 7.

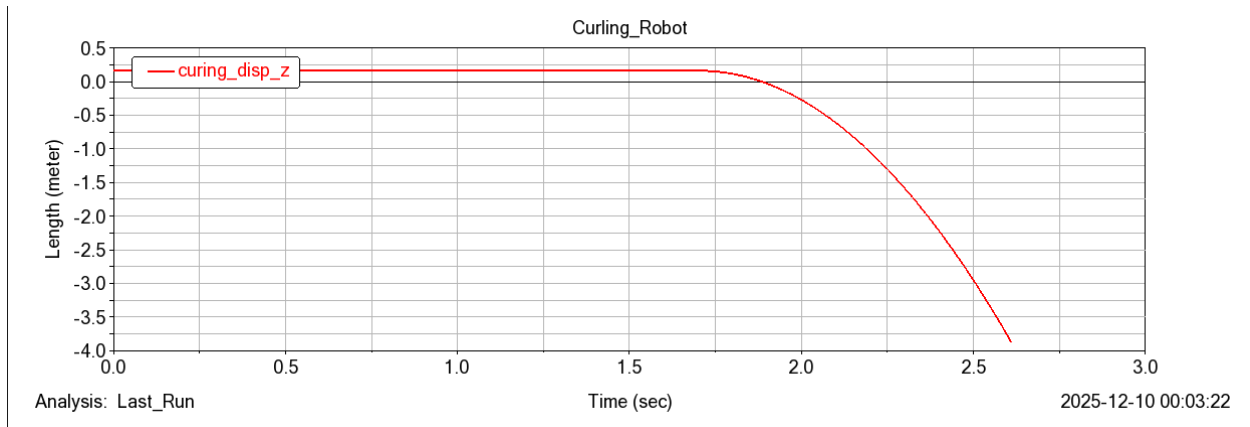


Figure 8: Curling stone displacement in Z

The displacement of curling stone in Z is -3.877m is shown above Fig 8.

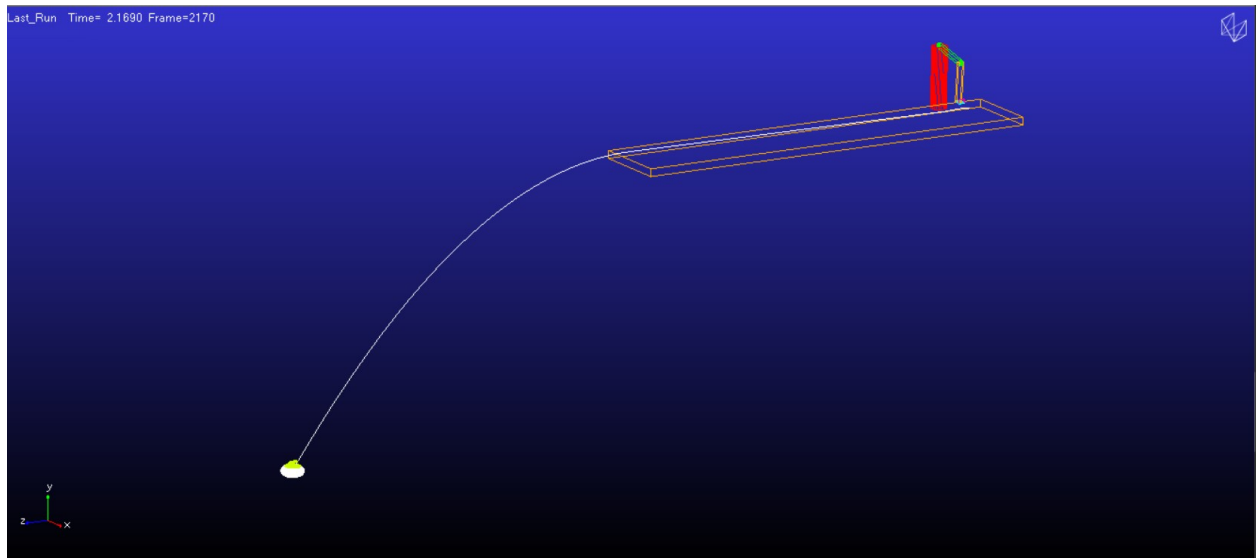


Figure 9: Trajectory of curling stone

The trajectory of the curling stone centre of mass is plotted and the results is as shown above Fig 9.